

Homework 1: Rasterizer

CS184/284A, Fall 2026 | Team nineteen

<https://cal-cs184-student.github.io/hw1-nineteen-writeup/>

Overview

This renderer draws SVG triangles, applies transforms, blends colors, and maps textures. Supersampling helps with jagged triangle edges. Bilinear sampling and mipmaps help with texture detail. The screenshots show why these need different kinds of sampling.

Task 1: Drawing triangles

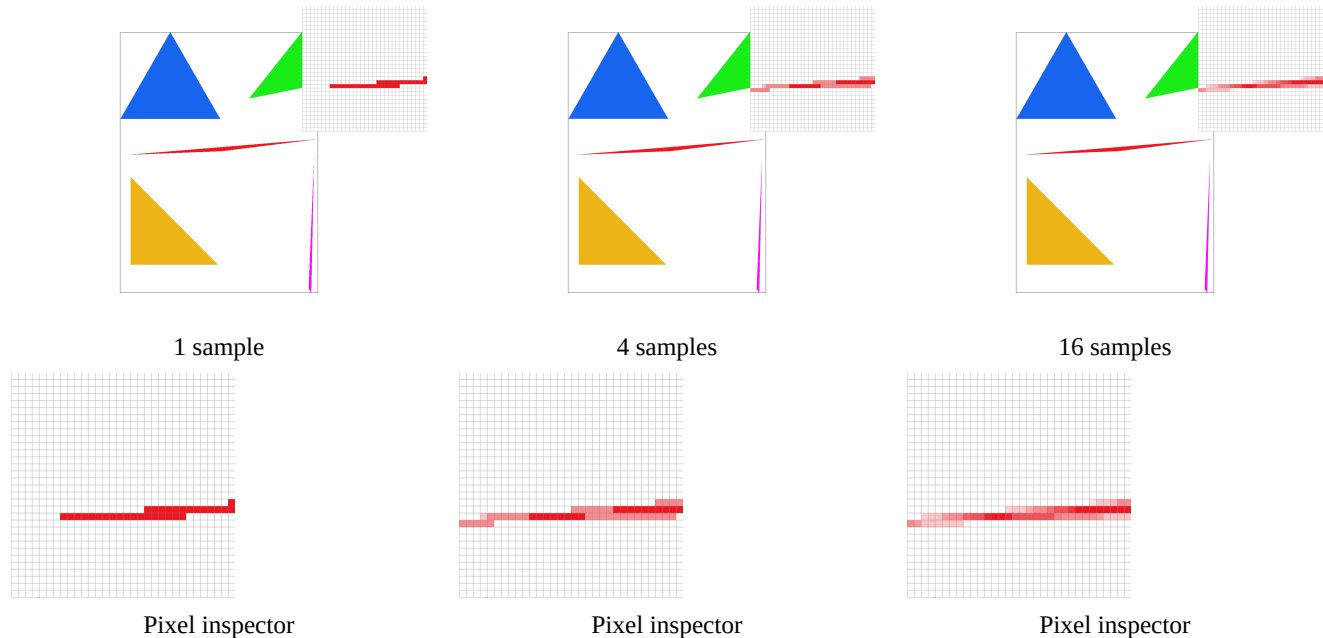
The code finds the smallest box around the triangle and clips it to the screen. It checks each pixel center at $(x + 0.5, y + 0.5)$. Three edge tests tell which side of each edge the point is on. All nonnegative or all nonpositive results mean it is inside. This includes edges and supports both vertex orders. Zero-area triangles are skipped.

Each sample takes three edge checks. Only the bounding box is visited, so the work is proportional to the samples in that box. The 1-sample image below is the Task 1 result.

Task 2: Supersampling

For s samples per pixel, the code uses a $\sqrt{s}$ by $\sqrt{s}$ grid. It tests the center of each small square and stores the covered colors in `sample_buffer`. The buffer has `width * height * s` entries, with each pixel's samples stored together. It is resized when the window or sample rate changes and cleared to white each frame.

Triangles write individual samples. Points and lines fill all samples of their pixel. After all shapes are drawn, `resolve_to_framebuffer` averages each pixel's samples into one RGB color.

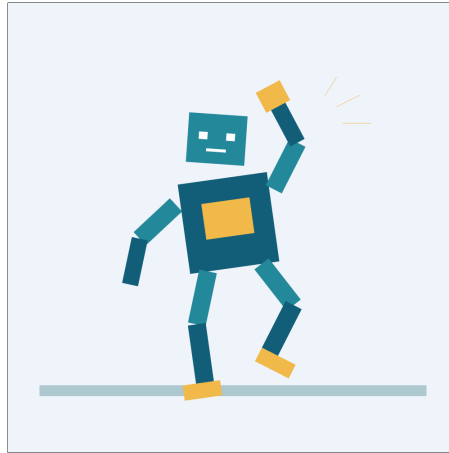


All three show `basic/test4.svg` at the default view with the same inspector position. At 1 sample, the red tip has sharp steps. At 4 and 16 samples, partly covered pixels become lighter red. More samples give smoother edges because they estimate coverage more closely.

Task 3: Transforms

Translation moves a shape, scaling changes its size, and rotation turns it. These use 3 by 3 matrices. Rotation converts degrees to radians before using sine and cosine.

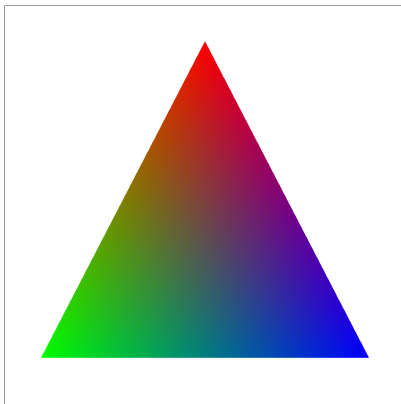
The robot is waving and stepping. The body tilts, one arm bends upward, and one knee bends. Nested groups put each joint in its own local coordinates. The forearm is inside the upper-arm group, so it follows the shoulder rotation. The file is my_robot.svg.



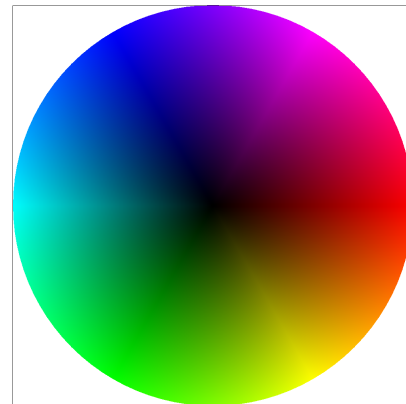
my_robot.svg, 16 samples per pixel.

Task 4: Barycentric coordinates

Barycentric coordinates are three vertex weights that add up to 1. Each weight is the area of the opposite small triangle divided by the full triangle area. The sample color is $a \cdot \text{colorA} + b \cdot \text{colorB} + c \cdot \text{colorC}$. At a vertex, only its color contributes. At the center, all three contribute equally.



RGB triangle, 1 sample



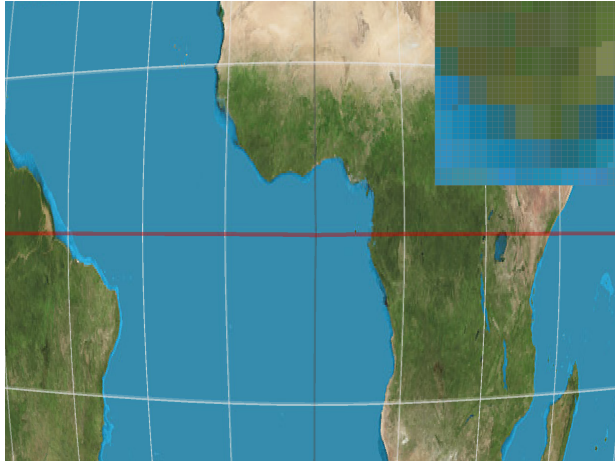
basic/test7.svg, default view, 1 sample

In the left triangle, the top is red, the bottom-left is green, and the bottom-right is blue. Points near the top have a larger red weight. Along the bottom edge, only green and blue are mixed.

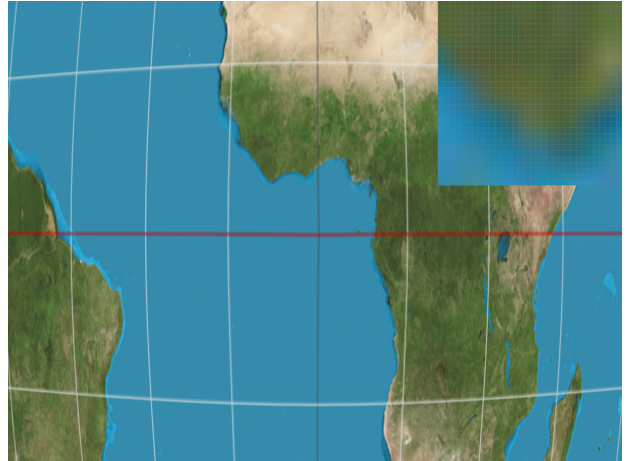
Task 5: Pixel sampling

Pixel sampling chooses a texture color. The code uses barycentric weights to find UV, then maps UV to $(u*(width-1), v*(height-1))$. Indices are clamped at the image edge.

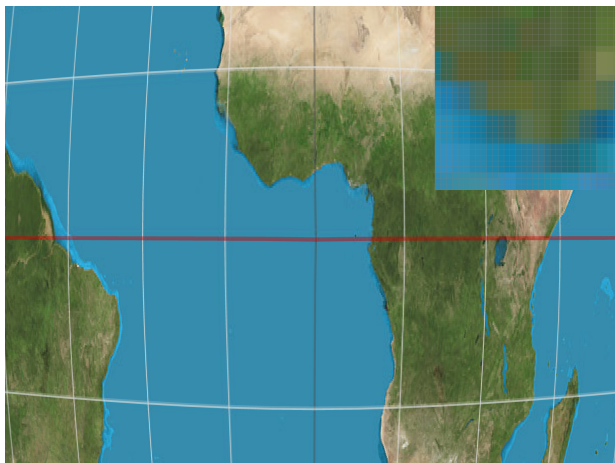
Nearest sampling rounds to one texel. Bilinear sampling blends the four nearby texels using the position between them. It first blends each row, then blends the two row results.



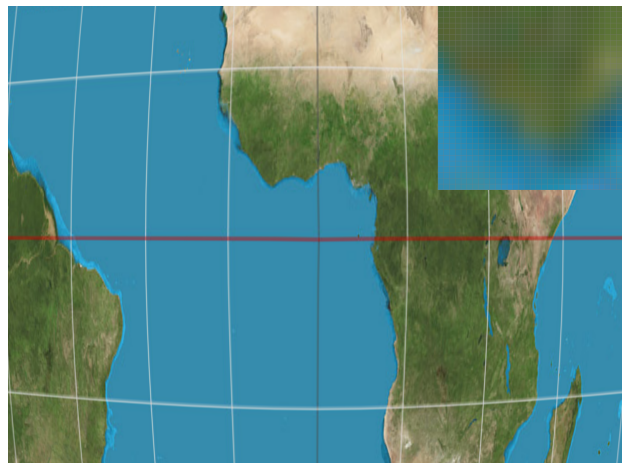
Nearest, 1 sample



Bilinear, 1 sample



Nearest, 16 samples



Bilinear, 16 samples

These use `texmap/test1.svg` with `L_ZERO`, the same zoom, and the same inspector position. Nearest at 1 sample looks blocky near the coast. At 16 samples, the edges of those blocks are smoother. Bilinear is already smooth at 1 sample, so 16 samples make a smaller difference.

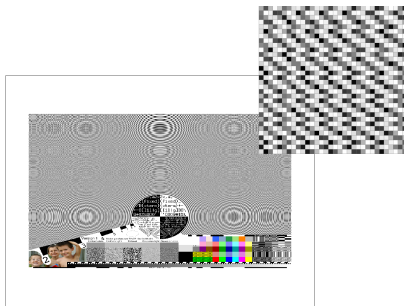
The difference is large when zoomed in on sharp texture changes. One texel covers many screen pixels, so nearest repeats blocks of color while bilinear blends between them. When zoomed far out, bilinear alone can still alias.

Task 6: Level sampling

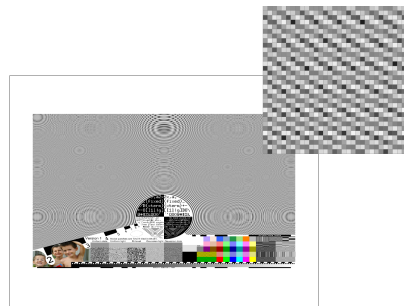
Mipmaps are smaller, averaged copies of a texture. The code finds UV at (x,y) , $(x+1,y)$, and $(x,y+1)$. It subtracts the first UV, then scales the u changes by the original width and the v changes by the height. The longer change gives the texture size covered by one screen pixel. Its $\log_2$ gives the mip level, clamped to the available levels.

L_ZERO uses level 0. L_NEAREST rounds to the closest level. L_LINEAR blends the two surrounding levels. Pixel sampling is chosen separately. Bilinear pixels with linear levels is trilinear filtering.

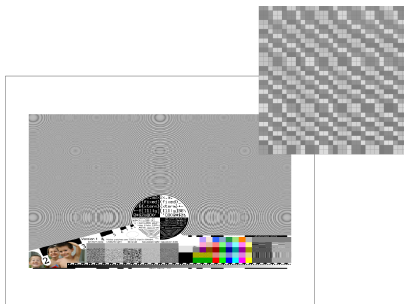
More screen samples smooth edges but increase shading work and sample memory. Bilinear reads four texels instead of one, needs no extra image storage, and helps enlarged textures. Mipmaps help small textures but use about one third more memory for a square power-of-two image. Linear level sampling reads two levels and gives smoother level changes.



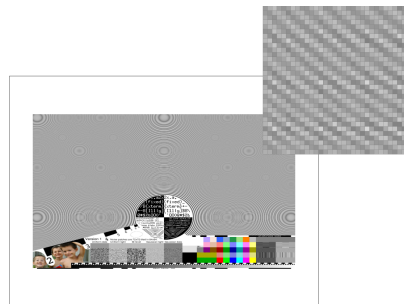
L_ZERO + P_NEAREST



L_ZERO + P_LINEAR



L_NEAREST + P_NEAREST



L_NEAREST + P_LINEAR

These use Zone plate boys at 1 sample with the same view. Level 0 shows false patterns. Bilinear reduces them a little. The nearest mip level removes more fine detail, and adding bilinear looks smoothest. The chosen mip level is 1 because the footprint is 1.8 texels per screen pixel.

A bug left the last mip levels black. Fixing the loop to fill every level solved it.

Known reference differences: a few thin edges and color-wheel pixels differ. The starter does not handle the partial stroke opacity in the lion and sphere.

Texture: AlbertCahalan, Zone plate boys, CC BY-SA 3.0. The original PNG and filtered screenshots use this license. Codex helped with code, tests, and writing.

Texture source | CC BY-SA 3.0 license